

Conv-ChArT — conference technical notes

Every sticky implementation detail, section by section

2026-07-29

Companion to `SLIDES.md`. Everything here is measured or read from source in the session that produced this file. Where a number is *inherited* from an earlier design-phase benchmark rather than re-measured, it is marked [design-phase].

1. Identity

Conv-ChArT (Convolutional ChArUco Transformer). Detects and identifies all 16 inner corners of a 5x5 ChArUco board (`cv2.aruco.CharucoBoard((5,5), 1.0, 0.7, DICT_5X5_50)`) and recovers board pose. Target application: outdoor module-to-module docking of a barrier robot under low light with active 940 nm illumination, where the input is a **lit-minus-unlit differenced frame**.

Three stages: **(1)** dense detector, **(2)** sub-pixel refiner, **(3)** symbolic lattice gate + recovery + PnP. Stage 3 is pure pipeline code, deliberately outside the exported graph.

2. Architecture and size

Trained variant: input **640 x 480**, `attend_div = 8`.

component	parameters	share
e1	9,632	0.2%
e2	55,552	1.2%
e3	221,696	4.7%
e4 (incl. dilated 2/4 pair)	2,066,432	44.0%
attention blocks (2x)	1,579,520	33.6%
gate3	24,705	0.5%
d3	442,624	9.4%
d2	110,720	2.4%
d1	27,712	0.6%
heatmap head	9,281	0.2%
class head	149,648	3.2%
detector total	4,698,034	4.70 M
refiner (separate checkpoint)	97,056	0.097 M
TOTAL	4,795,090	4.80 M

- Encoder: paired 3x3 conv-BN-ReLU blocks (milesial/U-Net shapes), with a **dilated (2, 4) pair** at the final encoder stage to widen RF cheaply before the bottleneck.
- Bottleneck: **2 pre-norm transformer blocks** (`timm vision_transformer.Block` shape), **8 heads**, **d = 256**, MLP ratio 4, final LayerNorm (eps 1e-6).
- Tokens: pure reshape of the H/8 map, **60 x 80 = 4800 tokens**, row-major. No patch-embed, no CLS, no absolute positional encoding.

- Decoder: bilinear upsample + concat, d3 (H/8 $\rightarrow$ H/4), d2, d1. **Only the H/4 skip is gated**; d2 and d1 skips are ungated.
- Heads: heatmap at **full resolution** (1 channel), class map at **H/4** (16 channels). Final logit-conv biases initialised to **-2.19** ($\pi = 0.1$).

Compute [design-phase]: ~ 84 G MACs, attention $\sim 37\%$ of them; forward B=1 **2.69 ms** (371 fps) on an RTX 5090 at bf16 + channels_last. **833 GFLOPs/frame** measured.

3. 2D axial RoPE

Rotates **Q and K only** (V unrotated), per head, so the attention dot product depends only on content and relative offset ($\Delta\text{row}, \Delta\text{col}$) — translation-equivariant by construction.

Per axis, $n = d_h/4 = 8$ frequencies, spaced **geometrically in wavelength** rather than in the standard base-exponent form:

$$\omega_i = \frac{2\pi}{\lambda_i}, \quad \lambda_i \text{ geometric on } [2.5, 2\max(H', W')] \text{ cells}$$

with $H' \times W' = 60 \times 80$ (8 px cells). Two reasons this departs from a textbook **rope-vit** lift, both worth knowing if challenged:

1. Standard RoPE’s fastest wavelength is $2\pi \approx 6.28$ cells for **any** base, so a “span from 2 cells” requirement is unsatisfiable by the textbook parameterisation. Wavelength-anchoring hits it directly. 2.5 rather than exactly 2.0 clears the Nyquist-degenerate sign-alternation at $\lambda = 2$.
2. $\lambda_{\max} \geq 2\max(H', W')$ makes the **slowest** pair’s phase injective over every in-frame offset, so the joint 8-frequency phase code never repeats for two distinct in-grid positions. Locked by `tests/test_model.py::test_rope_no_global_alias`.

Phase buffers are fp32. Aliasing of the *fast* pairs is inherent and desired — the vernier principle; the guarantee is on the ensemble, not per-pair.

4. The attention gate (exact formulation)

Oktay additive gate (Attention U-Net, eq. 1–2), applied to the **H/4 skip only**, conditioned on the post-attention bottleneck.

$$\alpha = \sigma\left(\psi\left(\text{ReLU}(W_x * s_{\downarrow 2} + W_g * g)\right)\right), \quad \tilde{s} = s \odot \text{up}_{\text{bilinear}}(\alpha)$$

- W_x : `Conv2d(skip_ch, inter_ch, kernel=1, stride=2, bias=False)` — stride-2, so the skip is downsampled to the gating signal’s resolution
- W_g : `Conv2d(gate_ch, inter_ch, kernel=1)`
- ψ : `Conv2d(inter_ch, 1, kernel=1)`
- α bilinearly upsampled back to skip resolution (`align_corners=False`), applied elementwise
- Channels at the trained variant: `AttnGate(skip=128, gate=256, inter=64)`

Pass-through initialisation — a real design decision, worth a sentence on stage. ψ ’s weight is zeroed and its **bias set to +3.0**, so

$$\alpha \equiv \sigma(3) \approx 0.953 \quad \text{exactly constant at step 0.}$$

Zeroing the weight (not merely biasing it high) is what makes the gate a **true no-op** at initialisation: a large bias alone would leave α technically input-dependent, so the network would start by fighting a small residual gating signal. Training then decides when gating begins. The Oktay reference repo’s TORR variant uses exactly $\psi.\text{bias} = +3.0$; the zeroed weight is our strengthening of it.

Ablation lever: `gates_enabled: false` bypasses the gate but still *constructs* the module, so `state_dict` shapes are unchanged and a gated checkpoint stays loadable — ablation by bypass, not by deletion, keeps parameter counts comparable.

5. Loss formulation

5.1 Targets

Heatmap target Y at full resolution: per-corner Gaussians, **combined by max** (never sum, per CornerNet), with the `rint( $p$ )` pixel **forced to exactly 1.0** so the $Y = 1$ branch is always reachable.

$$Y(u) = \max_{i \in \text{visible}} \exp\left(-\frac{\|u - p_i\|^2}{2\sigma^2}\right), \quad \sigma_{\text{hm}} = 2.0$$

Class target: same construction per channel at $H/4$, $\sigma_{\text{cls}} = 1.0$, cell coordinate $x_c = (x + 0.5)/4 - 0.5$.

5.2 Penalty-reduced focal, in logit space

One function serves both heads. $z = \text{logit}$, $\hat{p} = \sigma(z)$, $\alpha = 2$, $\beta = 4$:

$$\mathcal{L} = - \sum_u \begin{cases} (1 - \hat{p}_u)^\alpha \log \hat{p}_u & Y_u = 1 \\ (1 - Y_u)^\beta \hat{p}_u^\alpha \log(1 - \hat{p}_u) & \text{otherwise} \end{cases}$$

Implemented via `logsigmoid` — the naive $\log(1 - \hat{p})$ form **NaNs under bf16** near the forced peaks.

5.3 Normalisation — the detail that matters

$$\mathcal{L}_{\text{total}} = \frac{\mathcal{L}_{\text{hm}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}}{\max(\sum_{\text{batch}} N_{\text{vis}}, 1)}, \quad \lambda_{\text{cls}} = 1.0$$

- **One shared normaliser** for both heads: the batch total of visible corners. Per-channel $N \in \{0, 1\}$ would inflate $\mathcal{L}_{\text{cls}}$ by roughly $16\times$.
- Clamping at 1 handles all-negative batches ($N = 0$) with no special case — CornerNet’s own convention.
- Element counts are exactly equal on both sides at the trained variant (307,200 each), which is why $\lambda_{\text{cls}} = 1.0$ is a defensible default rather than a tuned constant.

5.4 Refiner loss

Same focal form on a **64 x 64 target at 8x** over the crop’s central 8 x 8 px, $\sigma = 1.5$, peak forced at `rint( $u^*$ )`. Encoding:

$$u^* = 31.5 + 8d, \quad d \in [-3.9375, 3.9375] \text{ px}$$

Decode is a 5x5 soft-argmax around the hard argmax, border-clamped, renormalised — in pipeline code, **not** in the `nn.Module`.

6. Stage 3, thresholds and constants

constant	value	meaning
<code>tau_hm</code>	0.30	heatmap peak threshold
<code>tau_id</code>	0.50	class-map confidence threshold
<code>lattice_tol_px</code>	3.0	homography inlier tolerance
<code>refine_min_peak</code>	0.30	refinement guard (below)

constant	value	meaning
<code>sigma_hm / sigma_cls</code>	2.0 / 1.0	target widths
<code>scale_range_px</code>	[12, 128]	trained board-square envelope
NMS radius	2 px	<code>merge_close</code> duplicate suppression
top-K peaks	64	decode cap
match radius	4.0 px	evaluation NN-match

Decode chain: peaks (3x3 max-pool equality) → `merge_close` → 24x24 sensor crops → refiner → soft-argmax → ID read → undistort → lattice gate → recovery → PnP (IPPE, `solvePnPGeneric`).

Half-pixel convention everywhere: $x' = (x+0.5)s - 0.5$ for every resolution change. The class map is read with `grid_sample(mode='bilinear', padding_mode='border', align_corners=False)` — `align_corners=True` is wrong here and biases by 0.375 cells (1.5 input px).

6.1 Two pipeline decisions worth defending

Identity is decoupled from refinement. The *entire* identity chain — class-map read, `lattice_gate`, and `recover` — runs on **coarse** coordinates. The refiner only supplies the reported position and PnP input. Verified **bit-identical**: 2054/2054 per-detection IDs match between the refined and coarse arms, 0 mismatched frames. Rationale: the class map is at H/4, so one cell is 4 input px and sub-pixel carries no information for identity; and `lattice_gate/recover` reassign IDs on the strength of *positions*, which is how the refiner was silently rewriting identity before this change.

Refinement guard. A corner is refined only if the refiner’s own peak sigmoid ≥ 0.30 ; otherwise it keeps its coarse position. Measured over 6,421 corners: the refiner is **worse than the coarse peak on 16.3% of crops**, and those crops are 55% featureless (crop std < 15, against 20% of the rest) — no amount of training recovers a sub-pixel position from a crop that does not contain a visible corner.

arm	median (px)	p95 (px)
coarse only	0.4230	0.8188
refined, no guard	0.0930	1.7939
refined + guard	0.0934	0.7419

The guard makes the refiner **better than coarse on both axes** — 4.5x the median accuracy *and* a better tail — which it was not before. Threshold swept across four candidate signals and 18 thresholds; image-statistic guards (crop std, saturated fraction) all lost to the network’s own confidence, and `sigma_px` never fired.

7. Results to have at hand

7.1 Four-system comparison (17 factors, identical frames, n = 100/step)

Mean recall across each factor’s swept range, worst step in parentheses:

factor	ours (coarse)	Deep ChArUco	classical
distance	97.6 (93.5)	83.2 (42.7)	30.6 (0.6)
brightness	96.5 (91.0)	82.5 (74.1)	30.3 (19.3)
ink_contrast	96.4 (94.7)	76.0 (58.5)	26.3 (11.3)
motion_blur	97.9 (95.9)	84.6 (76.8)	28.8 (12.5)
occlusion	98.0 (96.6)	85.6 (82.7)	34.1 (25.1)
tilt	97.5 (94.4)	92.8 (90.0)	53.8 (51.6)

Across all 17: ours **96.4–99.3%**, Deep ChArUco **76.0–92.8%**, classical **26.3–53.8%**.

Two caveats to state, not hide. Deep ChArUco is run **coarse-vs-coarse** — their RefineNet is not run, so their localisation belongs against our *coarse* arm. And their network sees 320x240 where ours sees 640x480: a real, reportable asymmetry, and part of why their distance curve falls off earlier. Classical’s ID accuracy is ~100% by construction (it only reports corners it has already identified) — its failure mode is **recall**.

7.2 Scale generalisation (outside the trained [12, 128] envelope)

ID accuracy, production checkpoint:

s (px)	6	8	10	12	16	64	128	160	192	256
ID %	0.0	60.0	80.9	90.5	96.5	100.0	99.8	86.5	37.8	5.6

Graceful to **s = 10** (2 px below the trained floor), cliff at 8, dead at 6. Upward, holds to **s = 160** (25% past the ceiling) then collapses by 192. Recall stays 85–97% at the top end — corners are still *found*; identity is what fails.

7.3 A1 — does attention earn its place?

One key (attn_blocks: 0), equal 50 k budget, identical frames.

regime	with attention	conv-only
far (s=12)	96.19%	97.52% (+1.34)
far (s=16)	93.98%	94.42% (+0.44)
very close (s=96)	99.73%	98.91% (-0.82)
very close (s=128)	99.20%	91.82% (-7.39)
beyond envelope (s=160)	82.9%	52.3% (-30.6)
occluded (max holes)	96.64%	96.32% (-0.32)
occluded (max objects)	96.88%	95.92% (-0.96)

Localisation is untouched — m01 identical to the fourth decimal (0.4232 vs 0.4234 at matched step 40 k). Attention sits at the bottleneck; the heatmap decodes at full resolution.

Conv-only was stopped at 41 k on demonstrated convergence (six consecutive validations within +/-0.14 pp against a 0.4 pp criterion; m01 flat in the fourth decimal over 15 k steps). Say “converged”, not “stopped early”, and have the numbers ready.

7.4 Attention head specialisation

40 frames, trained model, query = board-centre token. **Lift** = attention mass in a region divided by that region’s share of tokens (1.0 = uniform).

block 1	off-board mass	lift: corner	lift: marker
head 3	7.8%	14.1	22.0
head 5	4.3%	19.9	20.0
head 0	38.4%	22.8	7.9
head 6	41.6%	23.8	4.3

- **Block 0: zero marker readers.** All 8 heads junction-dominant.
- **Block 1: exactly two heads stay on the board** — 3 and 5, at 7.8% and 4.3% off-board mass, against 27–82% for every other head.
- **Head 3 preferentially reads code cells:** 22.0x marker enrichment vs 14.1x corner.

Do not read head identity off the figure. The introspection panel shows one frame and one query token, and which head *looks* spread varies frame to frame; the per-panel normalisation also makes diffuse background leakage read as board focus. The count comes from the aggregate.

8. Training recipe

- **250 k steps**, batch 32 effective, bf16 + channels_last. Production checkpoint at **160 k**.
- AdamW, lr **3e-4**, cosine to floor **3e-6**, **1000-step warmup** (mandatory with transformers), weight decay **1e-4**, grad clip **5.0**, **EMA 0.999**.
- ψ bias and all norms **excluded from weight decay**.
- Validation every 2.5 k steps on a fixed 2 k subset; full 10 k validation every 25 k.
- Data generated **on the fly** — no fixed training set. Loader: **spawn** context, 12 workers, **persistent_workers**, **prefetch_factor** 4, **cv2.setNumThreads(1)** in worker init. **Fork context loses 7x throughput** (55 vs 404 samples/s) — pinned in code, not config.
- Resume bumps a **resume_count** folded into the stream seed, so a resumed run does **not** replay the same sample sequence.
- Ablations: **50 k steps per arm**, one shared reference run, **exactly one config key** different per arm, verified programmatically before launch.

9. Deployment

- **Target: NVIDIA Jetson AGX Orin.** TensorRT fp16/INT8 engines exported from the bf16-trained checkpoint. No quantisation during development — all training and evaluation at full precision on an RTX 5090.
- **Budget:** 15 Hz pose = 30 fps of lit/dark differencing pairs = **66 ms per frame**.
- **833 GFLOPs/frame** measured. At realistic sustained Orin throughput (10–20 TFLOPS): **42–83 ms fp16** (marginal), **21–42 ms INT8** (comfortable).
- **Export parity gate — no engine is trusted until it passes:** corner error $\Delta < 0.05$ px and ID accuracy $\Delta < 0.1\%$ on 1 k validation images.
- **ONNX opset 17 via the TorchScript path** (dynamo=False; the dynamo path is broken in torch 2.8). Op set restricted to Conv / BN / ReLU / MaxPool / bilinear-Resize / Concat / Sigmoid / Slice / **DepthToSpace** — all TensorRT-standard, fp16-safe.
- **grid_sample** and all Stage-3 logic (peaks, NMS, lattice gate, recovery, PnP) live in pipeline code, deliberately **outside** the exported graph.
- **Sensor:** See3CAM_20CUG, OV2311 — 1600x1300 global-shutter mono, 3.0 μ m pixels, NIR-optimised for 940 nm. Centre-crop 1600x1200 $\rightarrow$ 640x480 gives $\rho = 2.5$. The sensor-to-input resize **must be antialiased** (INTER_AREA or equivalent prefilter): naive bilinear minification aliases the marker code past input Nyquist instead of blurring it.
- The refiner operates at **sensor** resolution, which is why it transfers unchanged between resolution arms.

10. Known limitations — have these ready

Being first to name them is cheaper than being caught by them.

1. **Motion blur beyond kernel 5 is out of distribution.** **motion_blur_kmax** = 5 in training; the robustness sweep tests 7 and 9. Both arms degrade there, and the refiner costs ~ 0.8 pp of recall at $k=9$ by displacing marginal corners past the 4 px match threshold.
2. **Single seed per ablation arm.** The A1 close-range effect is large and corroborated by two independent readings, but it is one seed.
3. **Uncertainty is not calibrated.** **sigma_px** is an *ordering* signal only — measured ~ 6 x over-confident as a variance. Do **not** describe it as a covariance or suggest wiring it into a filter's R .
4. **All results are synthetic.** The generator is calibrated to an OV2311-class industrial camera, but no real-frame evaluation exists yet.
5. **Deep ChArUco resolution asymmetry** (320x240 vs 640x480) is real and unmitigated — report it rather than letting it be discovered.